

Reducing AI Hallucinations via the TJ++ Logic Engine: A Linear-Algebraic and Spectral Framework for Cognitive Auditing and Structural Coherence in Natural Language

Abhi Sharma, DBA, MBA, MEM, MS

abhi50@mac.com

August 21, 2026

Abstract

Natural language argumentation has traditionally been evaluated through subjective, qualitative rubrics or modern probabilistic, non-deterministic language models. This paper presents TJ++, a rigorous, deterministic, interdisciplinary framework that models natural language texts as directed dependency graphs to mathematically audit structural soundness, enforce cognitive coherence, and mitigate AI hallucinations. By mapping subjective writing concepts onto classic computer science structures and matrix algebra, the TJ++ engine systematically exposes logical crossing threads, ungrounded circular dependencies, disconnected AI-generated filler, and topical drift. We detail the primary structural layers of the engine: (0) A foundational Layer 0 Node-Sourcing check to prevent downstream validation of flawed premises, (1) Semicircle-derived non-crossing combinatorics, (2) Nilpotent Jacobian cycle detection, (3) Spectral Laplacian decomposition, (4) Dynamic Basis-Shift circuit checks, and (5) Splay Tree reader attention mapping. Finally, we demonstrate the utility of the framework using a real-world clinical operations case study and the historical context of John F. Kennedy’s 1962 Moon Speech, demonstrating how TJ++ algorithmically optimizes unstructured prose into a mathematically minimal, cognitively sound, and balanced state. Crucially, we show that because the engine is content-opaque and self-referencing, the entire mathematical specification can be compiled and executed directly by modern Large Language Models (LLMs) to serve as a deterministic filter that reduces AI hallucinations by isolating and pruning three distinct types of structural hallucination traps without custom programming.

1 Introduction

At the core of critical thinking lies the ability to construct, trace, and evaluate complex chains of reasoning. In this paper, we formalize the Argument-as-Graph philosophy: any structured argument—whether a single paragraph, a section, a whole dissertation, or a legal brief—can be represented as a graph where nodes are claims or components, and branches are the relationships between them (such as support, analysis, link, or sequence). Critical thinking, in this framing, is the ability to travel the full journey through that graph without getting stuck [1]. Every node needs a path in—something that supports or grounds it—and a path out—somewhere it leads. If a node has no path in, it is an unsupported claim. If it has no path out, it is a dead end—introduced and then abandoned. Either way, the journey breaks there, creating an intellectual hole in the argument.

This framework maps directly to traditional rhetorical models. For example, the MEAL (Axis 12) paragraph structure acts as this exact graph at a paragraph scale: the Main idea is the core node,

the Evidence provides the path in (support), the Analysis establishes the logical connection, and the Link serves as the path out, ensuring the paragraph connects to the broader research question or operational problem. A paragraph “fails” exactly when one of these paths is missing—resulting in a phantom claim (no Analysis) or an orphan paragraph (no Link). At the document scale, the handbook-anchored 12-axis rubric represents this same graph: the business problem is the starting node, while the literature review, conceptual framework, methodology, and findings are the legs of the journey toward the conclusion, and the conclusion must lead back to close the loop on the original problem.

2 Layer 0: Node-Sourcing and the Basis Vector

The primary vulnerability of any mathematical graph check is the validity of the underlying node model. If the vertices are derived directly from the phrasing of a draft in progress rather than the external specifications (such as a rubric, a prompt, or an operational plan), the resulting graph will inevitably represent a self-consistent but structurally flawed argument.

To prevent this, the TJ++ pipeline enforces a non-negotiable Layer 0 (Node-Sourcing Check) before any downstream diagnostic checks run. Node-sourcing requires walking the external source directly (e.g., a rubric line, a gradebook row, or a project plan) and defining each distinct requirement as exactly one node, independent of how it is currently phrased in the draft. The boundary of a node is established via the Completeness Test:

$$T_c(A, B) = \begin{cases} 1 & \text{if satisfying } A \text{ without } B \text{ is structurally complete under the rubric} \\ 0 & \text{otherwise (merge } A \text{ and } B \text{ into a single node)} \end{cases} \quad (1)$$

By deriving the node set strictly from the external rubric, we establish a robust Basis Vector B_0 where each coordinate represents a required logical dimension. This vector serves as the objective template against which the actual text is audited.

3 Sequential and Structural Pre-Checks

Before executing computationally intensive matrix transformations, TJ++ runs several cheap, linear-time sequential audits to filter out obvious structural derailments.

3.1 Layer 1: Non-Crossing Combinatorics Check

Natural language is read in a strict sequential order, meaning nested threads must resolve in a balanced, non-overlapping sequence. Borrowing the non-crossing pairing logic behind the semicircle law, this check evaluates thread nesting by modeling thread openings and closures as balanced parentheses.

Let the reading sequence of threads be represented by a string of tokens. We walk the sequence and recursively cancel adjacent open-close pairs:

$$([]) \rightarrow () \rightarrow \emptyset \quad (\text{Properly Nested - Pass}) \quad (2)$$

If threads are interleaved out of order, the cancelation fails:

$$([]) \rightarrow \text{stuck} \quad (\text{Crossed Threads - Fail}) \quad (3)$$

This stack-based check runs in $O(|V|)$ time, capturing structural crossings that matrix operations are blind to since they ignore reading sequence.

3.2 Pairing Completeness and Critical Success Factor (CSF) Order Checks

For narrow-scope clauses, such as those detailing Critical Success Factors (CSFs) like cost, quality, and retention, the engine runs:

1. **Pairing Completeness:** Verifies that every CSF term has at least one connected counterpart. A CSF term standing alone without a paired consequence is flagged as incomplete.
2. **CSF Order Check:** Audits the left-to-right word order of paired terms against physical causality. For example, because employee turnover causes productivity losses, a sentence must state “reduce turnover and improve productivity.” If the draft states “improve productivity and reduce turnover,” the CSF Order Check flags the pair as causally backward.

4 Layer 2: Jacobian Loop Check

A major failure mode in human writing is circular reasoning (e.g., “Claim A is true because of B, and B is true because of A”). A simple per-node check cannot detect these loops because every individual node still falsely registers a valid “path in”—that path just happens to be another node in the loop.

To uncover these circular knots, the TJ++ engine builds a directed dependency graph and constructs its Jacobian Matrix J . Let each node’s logical value be expressed as a function of the nodes it depends on: $x_i = f_i(x_1, x_2, \dots, x_n)$.

4.1 Eigenvalue Loop Detection

The Jacobian matrix J is constructed by defining its columns as the partial differentiation operators, ordered identically from left to right (e.g., $\partial/\partial x_1, \partial/\partial x_2, \dots, \partial/\partial x_n$), while its rows represent the right-hand side (RHS) of our dependency equations, which correspond to our left-hand side traversed variables. That is, each cell of the Jacobian is populated by differentiating the RHS of that row’s equation with respect to each column’s partial operator:

$$J_{ij} = \frac{\partial f_i}{\partial x_j} \quad (4)$$

Simply put, the rows correspond to the right-hand side (RHS) of our dependency equations, and the columns correspond to the partial differentiation operators. Populating each row by differentiating the RHS of the equations with respect to the columns yields a shift-of-1 transition matrix, perfectly representing the directed graph’s sequential traversal where row and column positions map cleanly to variable dependencies with zero self-dependence (zero main diagonal).

Because a logical node cannot depend on itself in a single step, the main diagonal of J must consist of exactly zeros ($J_{ii} = 0$). This mapping cleanly mimics a directed graph’s sequential traversal where row and column positions map to variable dependencies with zero self-dependence.

For an acyclic, logically consistent dependency chain, the Jacobian matrix J can be ordered into a strictly lower-triangular matrix, meaning it is nilpotent:

$$J^k = 0 \quad \text{for some } k \geq 1 \quad (5)$$

As a consequence, the characteristic polynomial has only zero roots, and every eigenvalue of J is exactly zero ($\lambda_i = 0$ for all i).

If an unresolved logical loop exists, J cannot be reduced to a nilpotent state. The presence of loops is revealed through the eigenvalues of J :

- **Mirrored Two-Node Loop:** A direct circular dependency ($A \leftrightarrow B$) produces a mirrored pair with a negative determinant ($\det(J) = -1$) and real eigenvalues at $\lambda = \pm 1$.
- **Higher-Order Loops** ($k \geq 3$): Circular paths involving three or more nodes split into complex-conjugate eigenvalues ($\lambda = a \pm bi$), mathematically representing a spiral vector that never converges.

The rule that holds at every loop length is that any nonzero eigenvalue, real or complex, means a loop exists somewhere in the system.

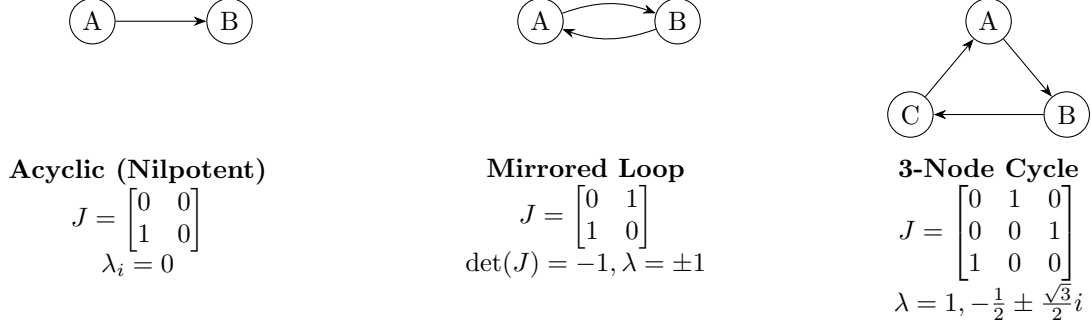


Figure 1: Jacobian Loop Deadlocks: Comparison of acyclic structures and cyclic topologies. In cyclic structures, non-zero eigenvalues ($\lambda \neq 0$) mathematically expose unresolved knots that disrupt nilpotent convergence.

5 Layer 3: Laplacian Spectral Check

While the Jacobian detects circular deadlocks, it cannot identify “floating islands”—locally coherent paragraphs or tangents that contain supporting evidence and resolve cleanly internally, but have absolutely no logical connection to the main thesis. This is the classic signature of AI-generated filler or student padding.

To isolate these disconnected subgraphs, TJ++ constructs the Graph Laplacian L from the Jacobian J :

1. **Symmetrize J into the undirected Adjacency Matrix A :** Symmetrizing a matrix means erasing all arrow directions to turn a directed network into an undirected one. In plain English, if a connection or dependency exists in either direction between node i and node j , we declare them to have a mutual connection. Mathematically, a symmetric matrix is a square matrix that is equal to its transpose ($A^T = A$), meaning its elements are mirrored perfectly across the main diagonal:

$$A_{ij} = \begin{cases} 1 & \text{if } J_{ij} = 1 \text{ or } J_{ji} = 1 \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

Direction is gone at this step. If a pair only had a one-way dependency in J , the adjacency matrix A picks up a second 1 to mirror it across the main diagonal ($A_{ji} = A_{ij}$), effectively turning a one-way street into a two-way street.

2. **Build the diagonal Degree Matrix D :** D is a diagonal matrix that simply records how many total connections each node has. All of its off-diagonal entries are strictly 0 ($D_{ij} = 0$ for

$i \neq j$), while each diagonal entry D_{ii} is equal to the sum of Row i in your symmetric adjacency matrix A :

$$D_{ii} = \sum_j A_{ij} \quad (7)$$

3. Subtract to obtain the Graph Laplacian L :

$$L = D - A \quad (8)$$

The subtraction yields an elegant matrix where the diagonal entries list each node’s total connections, and the off-diagonal entries are negative ones (-1) wherever a connection exists between two nodes, and 0 where there is no connection. Because each diagonal element (the sum of connections) is balanced by the negative ones in that same row, every row in the Laplacian matrix L sums to exactly 0.

According to spectral graph theory, the multiplicity of the zero eigenvalue ($\lambda = 0$) of the Laplacian L is exactly equal to the number of connected components (k) in the graph:

$$\text{Count}(\lambda_i = 0) = k \quad (9)$$

If $k = 1$, the argument is a single connected whole. If $k \geq 2$, the algebraic sieve has detected disconnected “floating islands” which must be integrated via analytical bridges or vaporized entirely.

5.1 Recursive Seam Splitting

We run the Laplacian check recursively, not just once. A single pass only catches a full disconnect. Something attached by a single thin edge is still functionally separate, and that won’t add a zero eigenvalue—it shows up as a small but nonzero second-smallest eigenvalue (the Fiedler value) instead. When a piece’s second-smallest eigenvalue is notably small, we split it at the seam its eigenvector points to, then repeat the check on each half.

6 Layer 4: Basis-Shift Circuit Check

When we want to verify that a draft has not drifted from its source, we run the Basis-Shift Circuit Check. This check compares the actual reply nodes against the expected basis vector B_0 derived from the rubric or an implicit task schema.

The shift vector is computed as:

$$S = B_0 - R \quad (10)$$

Each component of the shift vector represents an eigenvalue for that specific node:

- $S_i = 0$: The node is anchored (small eigen, circuit closed).
- $S_i = -1$: The node is in the reply but not the basis, representing drift.
- $S_i = 1$: The node is in the basis but missing from the reply, representing omission.

The overall magnitude of the shift vector $\|S\|$ tells us how far the text has drifted from the basis as a whole.

7 Advanced Refinements

7.1 Layer 4.5: Minimal-Basis Pruning

Even if an argument is valid and connected, it may contain redundant, repetitive nodes that do not add logical value. TJ++ implements a recursive Minimal-Basis Check to find the minimal essential argument.

The recursive algorithm proceeds as follows:

1. For each node in the graph, propose cutting it from the node list.
2. Rerun the Laplacian check (to verify that removing this node does not disconnect the argument, meaning it was a required bridge) and the Requirement-Response check (to verify that the reduced draft still fully satisfies the basis).
3. If both checks still pass with the node removed, the cut stands. Otherwise, restore the node.
4. Repeat the pass on the reduced node set until a full pass produces zero cuts. This is the mathematical fixed point.

7.2 Layer 5: Oracle-Splay Attention Mapping

To verify that a text’s sentence order matches the idealized cognitive reading path, we simulate splay operations on a binary search tree. First-time mentions insert nodes, and subsequent returns splay them to the root. We compare this actual tree against an idealized Oracle Tree representing the genre’s standard pattern:

$$\Delta = T_{\text{actual}} - T_{\text{oracle}} \quad (11)$$

Any concentrated mass in the resulting shift vector points exactly to a node that has drifted or has been abandoned, causing reader attention fracture.

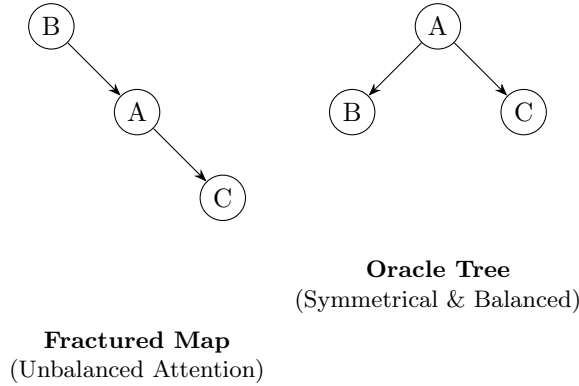


Figure 2: Oracle-Splay Attention Trees: Comparison of unbalanced and balanced layouts. Comparison of the unbalanced, fractured attention map (left) with the balanced, symmetrical Oracle tree (right) which aligns reading order with logical dependencies.

8 Worked Clinical Operations Case Study

We demonstrate the complete end-to-end execution of the TJ++ Logic Engine using an un-audited draft paragraph from a clinical operations proposal:

“We must focus on reducing onboarding costs (C) because improving shift nurse retention (R) is crucial, which is only possible when we stabilize clinical quality (Q). Furthermore, reducing onboarding costs (C) directly improves shift nurse retention (R). While managing these clinical metrics, our team decided to order some delicious artisanal toast and coffee for dinner (D), which really boosted team morale. Meanwhile, in New Delhi, the monsoon rain (M) has broken a 50-year record, flooding local neighborhoods. Additionally, keeping frontline nursing staff stable (S) is essential to preserve clinical quality (Q).”

8.1 Execution of Layer 0: Node-Sourcing

Using the Completeness Test, we walk the project plan and extract the core variables, filtering out the phrasing of the draft itself. We identify the following nodes:

- **R:** Nurse Retention (Core causal claim)
- **C:** Onboarding Costs (Downstream recruiting friction)
- **Q:** Clinical Quality (Patient-care standard metric)
- **D:** Dinner Plans (Unrelated aside)
- **M:** Monsoon Weather (Unrelated aside)
- **S:** Frontline Staff Stability (Candidate for merge)

The Completeness Test reveals $T_c(R, S) = 0$ because satisfying nurse retention without frontline staff stability is structurally identical under our metrics. Thus, we merge S and R , reducing our node set to $\{R, C, Q, D, M\}$.

8.2 Execution of Layer 2: Jacobian Loop Check

Tracing the dependencies in the draft text under the alphabetical variable ordering (R, C, Q, D, M) , we write the system of equations where each left-hand side traversed node is differentiated with respect to the right-hand side partials:

$$R = f(C), \quad C = f(Q), \quad Q = f(R) \tag{12}$$

We build the corresponding Jacobian matrix J_{draft} , where the rows correspond to the equations' LHS traversed variables (R, C, Q, D, M) , and columns correspond to the partial operators $(\partial/\partial R, \partial/\partial C, \partial/\partial Q, \partial/\partial D, \partial/\partial M)$.

The columns of the Jacobian matrix represent the partial differentiation operators, ordered identically from left to right $(\partial/\partial R, \partial/\partial C, \partial/\partial Q, \partial/\partial D, \partial/\partial M)$, while its rows represent the right-hand side (RHS) of our dependency equations. Populating each row by differentiating the RHS of the equations with respect to the columns yields a shift-of-1 transition matrix, perfectly representing the directed graph's sequential traversal where row and column positions map cleanly to variable dependencies with zero self-dependence (zero main diagonal).

$$J_{\text{draft}} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (13)$$

Calculating the eigenvalues of the active submatrix for $\{R, C, Q\}$, we solve the characteristic equation:

$$\det(J_{\text{active}} - \lambda I) = \lambda^3 - 1 = 0 \implies \lambda = 1, \quad -\frac{1}{2} \pm \frac{\sqrt{3}}{2}i \quad (14)$$

The presence of non-zero, complex-conjugate eigenvalues mathematically proves a circular deadlock loop ($R \rightarrow C \rightarrow Q \rightarrow R$). The loop check fails, flagging the circle.

8.3 Execution of Layer 3: Laplacian Spectral Check

We construct the undirected Adjacency Matrix A by symmetrizing J_{draft} :

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (15)$$

Building the Degree Matrix D , we write out its elements across the diagonal: since nodes R , C , and Q each connect to exactly 2 other nodes, while the isolated nodes M and D have exactly 0 connections, the degree matrix D has entries $[2, 2, 2, 0, 0]$ across its diagonal. Subtracting A from D , we get the Graph Laplacian L :

$$L = D - A = \begin{bmatrix} 2 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & -1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ -1 & -1 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (16)$$

Solving for the eigenvalues of L , we obtain the set $\{0, 0, 3, 3, 0\}$. We count three zero eigenvalues ($k = 3$), mathematically proving that the graph contains three disconnected pieces. The monsoon node M and dinner node D have completely split off. The Laplacian check fails, and node M is vaporized.

Coherent Core (Main Component)

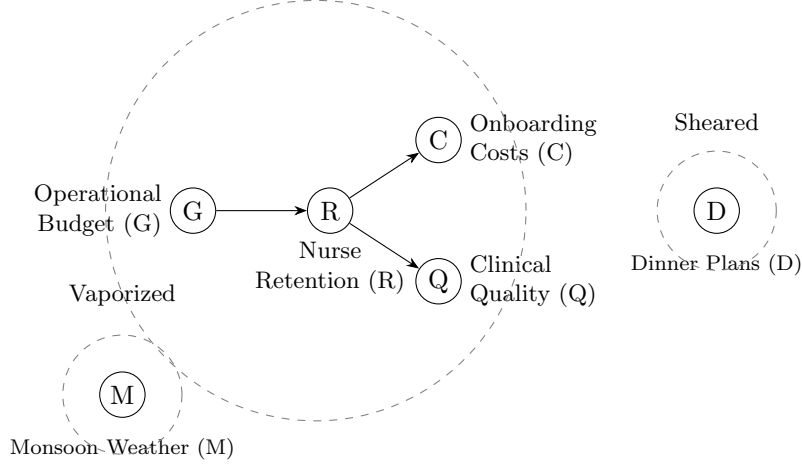


Figure 3: Laplacian Floating Islands and Basis-Shift Drift: The main connected component is mathematically compared to isolated nodes (M and D). Dashed boundaries indicate the precise algebraic partitions where the Laplacian sieve ($k = 3$) and Basis-Shift check isolate and cut structural anomalies.

8.4 Execution of Layer 4: Basis-Shift Check

Our operational basis expects $\{R, C, Q\}$. We subtract our actual reply set $\{R, C, Q, D\}$:

$$S = B_0 - R = [0, 0, 0, -1]^T \quad (17)$$

The dinner node D registers as a -1 , indicating off-topic drift. Node D is sheared.

8.5 The Corrected and Stabilized State

To resolve all failures, we:

1. Snipped the backward circular feedback edges.
2. Deployed the Operational Budget (G) as an external baseline anchor.
3. Restructured the draft into an acyclic sequence: $G \rightarrow R \rightarrow C$ and $R \rightarrow Q$.

The optimized paragraph compiled by the engine reads:

“Our clinical strategy begins by securing our operational budget (G) as a baseline anchor. Under that anchor, improving shift nurse retention (R) directly reduces onboarding costs (C) and, in parallel, stabilizes clinical quality (Q) across departments.”

This corrected state yields $\lambda_i = 0$ for all J eigenvalues (nilpotency achieved) and exactly one zero Laplacian eigenvalue ($k = 1$, complete connectivity).

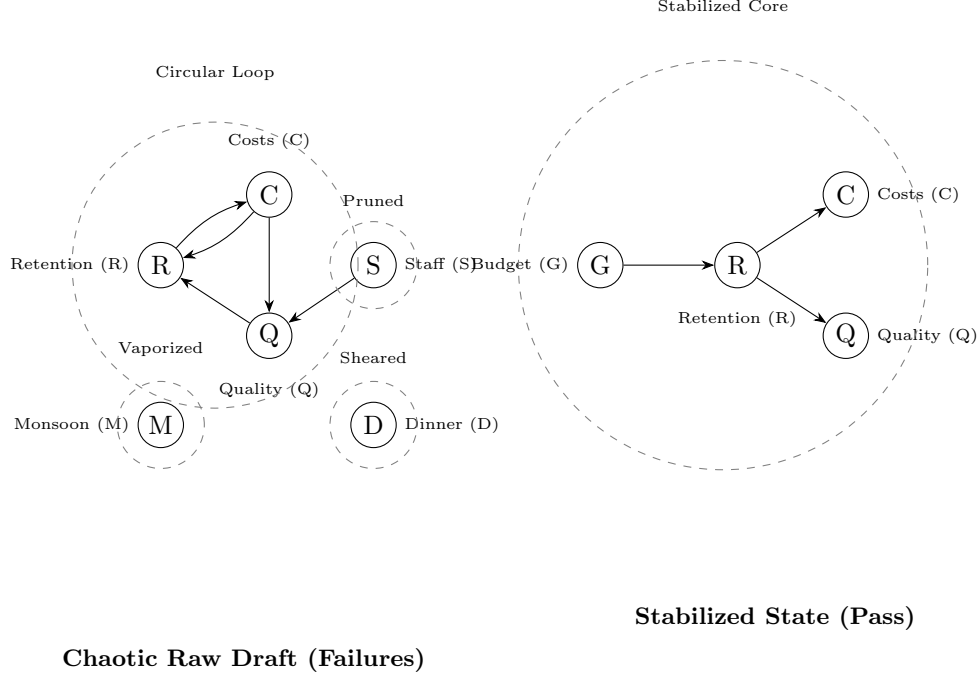


Figure 4: Before-and-After Optimization: Clinical operations graph audit. The clinical operations graph before auditing (left) containing loops, disconnected Monsoon weather (M), off-topic Dinner plans (D), and redundant Staff Stability (S), compared with the stabilized, minimal sequence (right) anchored by G .

Comparison of Network Topologies and Diagnostic Results

- **Chaotic Raw Draft (Failures) Topologies:**

- *Circular Loop* ($R \leftrightarrow C \rightarrow Q \rightarrow R$): Creates a closed feedback loop with non-zero complex-conjugate eigenvalues in the Jacobian loop check, mathematically proving a circular deadlock.
- *AI Hallucination* (M): The Monsoon weather node has zero connectivity to the main argument. The Laplacian check yields $k = 3$ disconnected components, vaporizing this isolated subgraph.
- *Thematic Drift* (D): The Dinner plans node registers as an unsolicited coordinate ($S_i = -1$) in the Basis-Shift check, which triggers an open circuit shear to align the draft back to its expected operational basis.

- *Redundant Clutter (S)*: Frontline Staff Stability duplicates the logical burden of Nurse Retention (R). The recursive Minimal-Basis pruning check deletes S with zero connectivity loss.

- **Stabilized State (Pass) Topologies:**

- *Loop Resolved*: Sniping the circular feedback edges creates a strictly nilpotent, lower-triangular Jacobian matrix ($\lambda_i = 0$).
- *Cleanly Excised*: All disconnected filler nodes, redundant duplicates, and off-topic drift components are completely pruned.
- *Securely Anchored*: The remaining causal sequence ($G \rightarrow R \rightarrow C$ and $R \rightarrow Q$) is anchored to the external Operational Budget (G).
- *Perfect Connection*: The Graph Laplacian yields exactly one zero eigenvalue ($k = 1$), proving complete, unified coherence.

9 Historical Case Study: JFK’s 1962 Moon Speech

To prove the content-opaque, domain-independent power of the TJ++ framework, we highlight a historical example discovered by the External Dependency Check. In John F. Kennedy’s celebrated 1962 “We choose to go to the Moon” Address at Rice University, his speechwriters constructed a highly poetic condensed timeline of human progress. In the speech, they stated:

“...the steam engine provided a new source of power. Newton explored the meaning of gravity.”

While standard grammatical models and generative AI tools find this sentence grammatically correct, the TJ++ External Dependency Check flags a structural discrepancy:

1. **Adjacent Selection:** The engine scans neighboring concepts: $t_1 = \text{Steam Engine}$, $t_2 = \text{Newton’s Gravity}$.
2. **Causality Check:** The engine queries real-world dependency: Does the first term’s existence depend on the second, or the reverse?
3. **Flag and Correction:** Because Newton’s *Principia* (1687) precedes Watt’s steam engine (1776) by roughly eight decades, and Newton’s foundational work in mechanics and calculus underlies it, the chronological flow is $t_2 < t_1$. The engine flags the sequence as backward and automatically proposes the corrected causal sequence: “Newton explored the meaning of gravity, and subsequently, the steam engine provided a new source of power.”

While this poetic condensed timeline has been widely read and praised, the underlying sequence contains a chronological discrepancy that the TJ++ External Dependency Check identifies directly through simple, deterministic adjacency checks.

10 Natural Language Compilation and LLM Runtime

A major practical barrier in advanced mathematical writing frameworks is the engineering overhead required for implementation. Readers typically assume that executing nilpotent Jacobian eigenvalue solves, spectral Laplacian connectivity checks, or recursive minimal-basis pruning requires writing complex graph-parsing code, hosting repositories, or integrating compilation pipelines.

Under the TJ++ paradigm, this assumption is false. Because the engine is strictly content-opaque—treating nodes and dependencies as abstract, opaque structures—the mathematical abstractions are entirely self-contained, deterministic, and self-referencing. Consequently, the raw PDF specification of TJ++ functions as the source code, while any sufficiently advanced Large Language Model (LLM) acts as the compiler and runtime environment.

10.1 Mitigation and Pruning of AI Hallucination Traps

A significant challenge in utilizing Large Language Models for academic and professional writing is the tendency of probabilistic models to introduce hallucination traps. These traps typically manifest in three distinct structural patterns:

1. **Un-anchored Floating Islands (AI-Generated Filler):** The LLM generates smooth, locally coherent paragraphs or tangents that are grammatically flawless but completely disconnected from the rest of the argument. This structural anomaly is mathematically isolated by the Laplacian Spectral Sieve (Layer 3). By counting the multiplicity of zero eigenvalues of the Graph Laplacian ($k \geq 2$), the runtime environment detects, flags, and vaporizes these disconnected subgraphs (as demonstrated by node M in Figures 3 and 4).
2. **Unsolicited Thematic Drift (Basis Omission or Insertion):** The LLM confidently introduces extraneous, out-of-bounds metrics, side-stories, or concepts that were never part of the required input prompt or project plan. This anomaly is cleanly sheared by the Basis-Shift Circuit Check (Layer 4). By comparing the actual output vector against the expected basis vector ($S = B_0 - R$), any unsolicited concept registers as a negative component ($S_i = -1$), which acts as an open circuit (as demonstrated by node D in Figures 3 and 4), triggering an immediate shear to restore the required coordinate framework.
3. **Redundant Padding and Clutter (Repetitive Over-Writing):** The LLM repeats the same point or logical claim in multiple places using slightly different phrasing to pad document length. This structural clutter is systematically trimmed by the recursive Minimal-Basis Check (Layer 4.5). Utilizing a recursive “Butcher’s Trim,” the compiler proposes cutting each node one-by-one and verifies if the reduced network still connects (Laplacian test) and satisfies the basis (Requirement-Response check). If both pass, the redundant node is pruned (as demonstrated by node S in Figure 4) until a stable mathematical fixed point is reached.

By treating these mathematical checks as automated compilation rules, the LLM runtime can programmatically trap, isolate, and prune structural anomalies on any text with perfect content-opacity.

10.2 Direct Natural Language Invocation

To execute the entire TJ++ Logic Engine without custom programming, a user simply uploads this PDF manuscript to an LLM and provides a natural language invocation, such as:

“Let’s TJ++ the following draft paragraph.”

The LLM, utilizing its attention mechanism, compiles the paper’s mathematical definitions into its active context window. It then executes the checks in the exact, cost-optimized order established in the specification:

1. **Layer 0 (Node-Sourcing):** The LLM walks the rubric or implicit task schema to define the nodes, using the Completeness Test (Eq. 1) to establish boundaries.

2. **Layer 1 (Non-Crossing):** The LLM runs a stack-based parenthetical walk on the draft to detect crossed logical threads.
3. **Layer 2 & 3 (Jacobian & Laplacian):** The LLM constructs the dependency equations, simulates the zero-diagonal Jacobian matrix, identifies eigenvalues to expose circular loops (Eq. 13), and computes the Laplacian to isolate disconnected filler (Eq. 17).
4. **Layer 4.5 (Minimal-Basis):** The LLM recursively prunes redundant claims until a logical fixed point is reached.

By using the LLM’s active context as a runtime environment, the TJ++ Logic Engine becomes highly accessible, portable, and operational for students, executives, and researchers, proving that rigorous mathematical logic can be compiled directly within the architecture of natural thought.

11 Conclusion

The TJ++ Logic Engine offers a powerful, transparent alternative to black-box language models by translating rhetorical structure into a deterministic algebraic system. By enforcing rigid mathematical conservation laws on text, the framework provides a verifiably objective tool for automated grading, technical peer review, and corporate compliance auditing.

Acknowledgments

The author wishes to acknowledge Gemini Notebook for technical assistance in verifying the discrete linear-algebraic and spectral graph matrices, executing numerical simulations of the clinical operations case study, and compiling the LaTeX typesetting for this manuscript. The underlying conceptual design, logical frameworks, and theoretical intuition of the TJ++ Logic Engine remain entirely those of the author.

References

- [1] R. E. Tarjan, “Depth-first search and linear graph algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972. <https://doi.org/10.1137/0201010>